

✓ Sum of Digits Using Recursion

```
def sumofdigits(n):  
    if n == 0:  
        return 0  
    return (n % 10) + sumofdigits(n // 10)  
  
print(sumofdigits(12345))
```

15

✓ Reverse a Number Using Recursion

```
def palindrome(m):  
    if m < 10:  
        return m  
    return (m % 10) * 10 + palindrome(m // 10)  
  
print(palindrome(121))
```

31

✓ Check Palindrome String Using Recursion

```
def palindrome(s):  
    if len(s) < 1:  
        return True  
    if s[0] != s[-1]:  
        return False  
    return palindrome(s[1:-1])  
  
print(palindrome("madam"))
```

True

✓ Find Power of a Number

```
def power(base, power):  
    if power == 0:
```

```
        return 1
    return base ** power

print(power(2, 3))

8
```

✓ Power Using Recursion

```
def power(base, exponent):
    if exponent == 0:
        return 1
    return base * power(base, exponent - 1)

print(power(2, 3))

8
```

✓ Student Class Object Creation

```
class Student:
    pass

s1 = Student()
s2 = Student()

print(s1 == s2)

False
```

✓ Mobile Details Using Class and Object

```
class Mobile:
    def __init__(self, storage, battery):
        self.storage = storage
        self.battery = battery

    def display_details(self):
        print(f"storage: {self.storage}, Battery: {self.battery}")

i11 = Mobile("6GB", "3000 mah")
i22 = Mobile(8, 5000)
```

```
i11.display_details()  
i22.display_details()
```

```
storage: 6GB, Battery: 3000 mah  
storage: 8, Battery: 5000
```

✓ Student Details and Age Comparison

```
class Students:  
    def __init__(self, name, age, gender, branch):  
        self.name = name  
        self.age = age  
        self.gender = gender  
        self.branch = branch  
  
    def display_details(self):  
        print(f"Name: {self.name}, Age: {self.age}, Gender: {self.gender}, Brar  
  
s1 = Students("Dudu", 21, "Male", "Mech")  
s2 = Students("Bubu", 21, "Female", "Mech")  
  
s1.display_details()  
s2.display_details()
```

```
Name: Dudu, Age: 21, Gender: Male, Branch: Mech  
Name: Bubu, Age: 21, Gender: Female, Branch: Mech
```

✓ Stack Implementation

```
class stackimplement:  
    def __init__(self):  
        self.stack = []  
  
    def push(self, item):  
        self.stack.append(item)  
  
    def pop(self):  
        return self.stack.pop()  
  
    def peek(self):  
        return self.stack[-1]  
  
    def is_empty(self):  
        return len(self.stack) == 0  
  
    def size(self):
```

```
        return len(self.stack)

    def display(self):
        print(self.stack)
```

✓ Queue Implementation

```
class QueueImplement:
    def __init__(self):
        self.queue = []

    def enqueue(self, item):
        self.queue.append(item)

    def dequeue(self):
        return self.queue.pop(0)

    def is_empty(self):
        return len(self.queue) == 0

    def size(self):
        return len(self.queue)

    def display(self):
        print(self.queue)
```

✓ Valid Parentheses Using Stack

```
class Solution:
    def isValid(self, s):
        stack = []

        for ch in s:
            if ch == '(' or ch == '{' or ch == '[':
                stack.append(ch)
            else:
                if not stack:
                    return False

                top = stack.pop()

                if ch == ')' and top != '(':
                    return False
                if ch == '}' and top != '{':
                    return False
```

```

        if ch == ']' and top != '[':
            return False

    return not stack

s = Solution()
print(s.isValid("{}"))

```

False

✓ Reverse a String

```

S = ")()("
print(S[::-1])

```

()()

✓ Min Stack Implementation

```

class MinStack:
    def __init__(self):
        self.stack = []
        self.min_stack = []

    def push(self, value):
        self.stack.append(value)

        if not self.min_stack or value <= self.min_stack[-1]:
            self.min_stack.append(value)

    def pop(self):
        if self.stack[-1] == self.min_stack[-1]:
            self.min_stack.pop()

        self.stack.pop()

    def top(self):
        return self.stack[-1]

    def getMin(self):
        return self.min_stack[-1]

```

✓ Queue Using Two Stacks

```

class MyQueueusingstack:
    def __init__(self):
        self.stack1 = []
        self.stack2 = []

    def push(self, x):
        self.stack1.append(x)

    def pop(self):
        if not self.stack2:
            while self.stack1:
                self.stack2.append(self.stack1.pop())

        return self.stack2.pop()

    def peek(self):
        if not self.stack2:
            while self.stack1:
                self.stack2.append(self.stack1.pop())

        return self.stack2[-1]

    def empty(self):
        return not self.stack1 and not self.stack2

```

✓ Count Ways to Climb Stairs (1,2,3 Steps)

```

def Steps(n):
    if n < 3:
        return n
    else:
        return Steps(n-1) + Steps(n-2) + Steps(n-3)

print(Steps(5))

```

11

✓ Find Nth Term of Arithmetic Progression Using Recursion

```

def AP(a, d, n):
    if n == 1:
        return a
    else:

```

```
        return AP(a, d, n-1) + d
```

```
print(AP(2, 1, 3))
```

```
4
```

✓ Check Divisibility of a Number

```
n = int(input())

if n % 2 == 0:
    print("Divisible by 2")
elif n % 2 == 0 and n % 3 == 0:
    print("Divisible by 6")
elif n % 5 == 0:
    print("Divisible by 5")
elif n % 5 == 0 and n % 10 == 0:
    print("Divisible by 10")
elif n % n == 0:
    print("Divisible by itself")
else:
    print("None of the above")
```

```
13
Divisible by itself
```